M2: Agile Requirements and Backlog Planning

1. Team Identity and Infrastructure

Member	Role	Responsibilities
Siddhu Bhimireddy	Design, architecture, code	All responsibilities

Git Repository: https://github.com/flufcloud/366_project
Agile Project Board:
https://github.com/users/flufcloud/projects/2/views/1
Shared Workspace: 366 Work Document

2. Project Vision and Scope

System Narrative

For the class project, I am creating a local Linux command-line tool designed to leverage cutting-edge AI and generative AI technology to help developers and security engineers understand, document, and evaluate the security risks and capacities of a codebase. The user points the tool at a repository, and the application will automatically generate structured, AI-generated documentation of the codebase. It then connects to the Github API and allows users to fetch open issues and pull requests, and performs analysis by piping the issues / PRs to language models to assess each item's security risk level (either low, medium, or high), and proposes preliminary investigations/root causes that can be of assistance to developers.

Minimum Viable Product (MVP)

Feature 1: Codebase Documentation Engine

The ability to recursively scan a repository and produce documentation that covers the high level data flow, architecture, and security considerations of the project.

Feature 2 Issue and PR analyzer:

Authenticates with Github and fetches open issues and PRs. It presents the user an interactive menu. For a selected item, the system will analyze it against the codebase and assign a risk classification with justification.

Feature 3 Preliminary Solutions:

For each analyzed issue or PR, the system will produce a plain-language suggested fix or direction of investigation to address the security problem.

3. Functional Requirements: User Stories and Use Cases

User Story	Acceptance Criteria	Security / Abuse Case
As a developer, I want to generate documentation for my repo so that I can quickly understand an unfamiliar codebase.	Docs must be generated in valid Markdown. Secrets (API keys, tokens) NEED to be redacted and flagged and never printed in plain text.	As an attacker, I should not be able to inject commands through malicious filenames or specially crafted source files (path traversal, shell injection).
As a security engineer, I want to select a Github PR and receive a risk classification so that I can prioritize my review queue.	The risk score must be classified properly, the terminal should display the output appropriately, and the Github token of the accessing user should never be revealed / vulnerable to the LLM.	As an attacker, I should not be able to exfiltrate the user's Github token by injecting payloads into PR titles or body text.
As a developer, I want to view a risk mitigation proposal for a flagged issue so that I know what code changes to make.	The proposal references specific files and line numbers. Suggestions must be syntax-valid for the detected language. No network calls are made beyond the initial Github fetch.	As an attacker, I should not be able to cause arbitrary code execution by crafting a malicious PR diff analyzed by the tool. The solution mitigation should not be a route by which I can do a prompt injection attack.

4. Non-Functional Requirements

Core Constraints

- Performance: The terminal application must be lightweight. It should not use more than 100,000 tokens of LLM API usage per request. The documentation recursively updating must be efficient and use tokens sparingly.
- Usability: All interactive flows must be navigable using keyboard only (arrow keys, Enter, Escape). Error messages must be human-readable and suggest a corrective action.
- Portability: The tool must run on a Linux system (common distros such as Ubuntu and Debian should work). A single install script must handle all dependencies. The user should have control over which LLM they want to use to power their system (ie Claude, Gemini, etc).

Security-First Design — Threat Model Basics

- Personal Assets: Github access tokens should be privatized, analyze source code safely while exposing as little underlying private data as possible.
- Primary Threats: Credential leakage via logs or stdout; code injection through maliciously crafted repository files; dependency confusion attacks via third-party packages; LLM prompt injections via PRs or issues; revealing private API data to an LLM provider (via prompting an LLM with confidential data on accident)
- Defensive Strategy: Github tokens and other private API keys (LLMs, etc) must be privately stored in local files. All inputs must be sanitized and ensured that no private confidential data is passed in before being sent to any LLM. Do not have code analysis, mitigation strategies, or other features of the system rely on sending private data to LLMs.

5. Usability and Reliability Requirements

Robustness Requirements

- The tool must reject bad inputs with a detailed error message and a list of accepted inputs/commands.
- If the API is unavailable, the tool must provide a Warning and stop executing, and provide the user directions to set their tokens and API keys up. There should be a --api-key-status flag to check for key validity.
- Do not print raw stacktraces. Instead, catch errors and print processed issues to the command line for user visibility and readability.

Availability and First-Run Experience

- A new user on a standard Linux distribution should be able to install the tool using only the README in under 5 minutes.
- The tool should run smoothly and be documented in such a way that common issues have easily visible fixes.

Support Documentation

- The README must cover prerequisites, installation, token setup, a quickstart command, all CLI flags, and a troubleshooting section addressing the common failure modes (incorrect dependencies, etc).
- A SECURITY.md must document how to report vulnerabilities and the tool's data handling guarantees, in particular a clear specification of how and what data the tool hands to proprietary LLMs.

6. Agile Process and Risk Management

Sprint Cadence

I will operate sprints of length one week. Sprint planning will ocur each Monday prior to class. I will review the final results of the sprint each Friday. The backlog of the issues/necessary features with respect to the MVP will be maintained in Github Projects, with updates happening at the beginning of each sprint.

Tooling and DevSecOps

Category	Tool	Justification

Language	Python 3.11	Python has many CLI libraries and also contains numerous libraries that support rapid iteration
Generative AI Scanning	LLM APIs	I need LLM APIs to access production grade frontier AI capabilities to accurately review PRs and issues for security vulnerabilities
Static Security Analysis	Bandit	I will use this for security tooling on the source code for this tool via static analysis
Testing	pytest	I will use this to test our code, I do not anticipate requiring further tooling for testing.
CI/CD	Github Actions	Github actions will be important to capturing the issues and PRs via Terminal, and allowing our tooling to interact with the repo.

Risk Backlog

Risk	**Description**	**Likelihood / Impact**	**Mitigation Plan**
Exposing private data	A team member might mistakenly commit a security token or API key to the repo.	Medium / High	We must never directly use tokens in code. Instead, we should refer to files.
Scope creep	The project scope can easily balloon past what is viable to create during our timeframe.	High / Medium	Focus on the MVP goal, and follow that heuristic strictly. Stretch goals or extra features should not distract the team from primary goals. Keep sprints focused on MVPs.
LLM/API Rate Limiting	Rate limiting.	Medium / Medium	Instead of testing the core functionality of pulling issues and organizing the information in the terminal and presenting to the user with a live LLM, use a temporary mock system. Only add the LLM when testing the security capabilities.